Algorithms & Planning: Toy Problem - Distributed excavating

Design a multi agent planning system, simulating a fleet of excavation robots.

The system consists of a 2D grid (see example below) with multiple excavation robots navigating this grid. Assume that the robots can dig dirt at various locations and must offload the dirt at predefined drop-off locations. The planner needs to compute motion plans for the robots to go to these dig locations, dig and then offload their load.

The planner has access to the grid (see starter code). New dig locations can be added to this grid, which the planner should monitor and act accordingly (if a new dig location is being created the planner should assign that job to a robot or eventually queue up work if all robots are busy). A robot can offload their payload at **any** drop off location. A robot can offload its payload when standing next to the drop off location (i.e., distance == 1 in x or y). A robot has only capacity for the load of one dig location.

We assume here that digging at any dig location takes the same amount of time no matter what location, and when the robot is done digging the dig location becomes a regular (non dig spot) afterwards. The robot moving in x or y dimension one step takes 1 tick. Also, the digging operation takes 1 tick.

If the planner has multiple dig locations in the queue it will produce plans so the robots can operate concurrently and work gets done faster. The planner avoids robot collisions but also ensures that robots can reach their destinations efficiently.

Example scenario with a 10x10 grid:

```
Unset
Initial grid:             -> becomes after sending 3   -> becomes after robots
                                dig locations:             worked off their jobs


R . # . . . # . . 0          R . # . . . # . . 0          . . # . . . # . R 1
. # . # . . . . . .          . # . # . . . D . .          . # . # . . . . . .
. . . . . . . . . .          . . . . . . . . . .          . . . . . . . . . .
. . # . . . . . . .          . . # . . . . . . .          . . # . . . . . . .
. . . . . . . . . .          . . . . . . . . . .          . . . . . . . . . .
. . . . . 0 . . . .          . . . . . 0 . . . .          . . . . . 2 . . . .
. # . . . . . . . .          . # . . . . . . . .          . # . . . R . . . .
. . . # . . . . . #          . . . # D . . . . #          . . . # . . . . . #
. . . . . . . . . .          D . . . . . . . . .          . . . . . . . . . .
. # . . . . . . . R          . # . . . . . . . R          . # . . . . . . . .
```

with:

```
Unset
. .. for empty cells
# .. for obstacles
R .. for robot positions
D .. for dig locations
0 .. for drop-off locations (number indicates how many loads this spot has
received in total)
```

Deliverables:

- Functions (especially "*monitor*") written out for the planner (ideally in C++).
- Instructions to build and run the code. Code must compile and run.
- **Concrete scenario in the starter code:**
    - As shown above, we are sending the 3 dig locations: (1,7), (7,4) and (8,0). After some amount of time all digging is done and the accumulated load of both drop off locations is 3.
    - A follow up scenario (as demonstrated in the starter code) could be that after some time a new dig location has been added and the planner needs to pick that work up.
- Use of generative AI tools (e.g., ChatGPT) is ok but needs to be precisely documented which parts in the code or system design are from such a tool.
- Explanation of chosen algorithms and a sense of runtime complexity.
- Think through cases of dealing with errors like the planner software crashing. Or robots crashing, or needing longer to dig. ← no code needed here; just some discussion topic for a follow up meeting.
- If we need to figure out what is an ideal number of robots to maximize throughput, how would you approach this question?